

pg_xid_wraparound Saturation Alert

Risk of DB shutdown in the near future, approaching transaction ID wraparound

This is a critical situation.

This saturation metric measures how close the database is to Transaction ID wraparound. When wraparound occurs, the database will automatically shut-down to prevent data loss, causing a full outage. Recovery would require entering single-user mode to run vacuum, taking the site down for a potentially multi-hour maintenance session.

To avoid reaching the db shutdown threshold, consider the following short-term actions:

1. Find and terminate any very old transactions (including those on replicas that have `hot_standby_feedback = on`). Do this first. It is the most critical step and may be all that is necessary to let autovacuum do its job.
2. Run a manual vacuum on tables with the oldest `relfrozenxid`. Manual vacuums run faster than autovacuum because `vacuum_cost_delay = 0` by default, so there it is not throttled.
3. Add autovacuum workers increasing `autovacuum_max_workers` (requires restart) and/or reduce `autovacuum_cost_delay`, if autovacuum is chronically unable to keep up with the transaction rate. Note, too many workers and too low `autovacuum_cost_delay` can saturate resources – first of all, we need to keep an eye on disk IO (read and write throughput, IOPS).

Finding and terminating very old transactions

Use the following query to find and review long-running transactions. Uncomment the commented line to terminate all long running transactions once you understand what they are.

```
select
    /* pg_terminate_backend(pid) as terminated, */ /* Uncomment this line to kill old transa
    now(),
    age(backend_xid) as xid_age,
    now() - xact_start as xid_age_in_wallclock_time,
    *
from
    pg_stat_activity
where
    backend_xid is not null
    and age(backend_xid) > 2^30 /* over half the max age */
order by
    xid_age desc;
```

More Reading

1. Official documentation: <https://www.postgresql.org/docs/current/routine-vacuuming.html#VACUUM-FOR-WRAPAROUND>
2. Internals: https://www.interdb.jp/pg/pgsql05.html#_5.10.1.

Others' experience when reaching the limit (good to understand how painful it is and what people do when it happens):

1. Sentry Transaction ID Wraparound Outage Incident Review: <https://blog.sentry.io/2015/07/23/transaction-id-wraparound-in-postgres>
2. Mailchimp Outage Incident Review: <https://mailchimp.com/what-we-learned-from-the-recent-mandrill-outage/>. Thread on the orange website.